Inventory Management System

Using MERN Stack

Step 1: System Analysis and Design

Scenarios, Use Case Diagram, Activity Diagrams,
Class Diagram and Database Diagram

Web Engineering Lab Project

Jashore University of Science and Technology

Contents

1 Introduction **2**

2 Scenarios **2**

2.1 Scenario 1: Walk-in sale 2

2.2 Scenario 2: Credit sale with partial payment 2

2.3 Scenario 3: Purchase from a supplier and supplier payment 2

2.4 Scenario 4: Damaged product and low-stock alert 3

2.5 Scenario 5: Customer return, and cancelling a wrong return 3

2.6 Scenario 6: Administrator adds a staff user 3

3 Use Case Diagram **4**

4 Activity Diagrams **5**

4.1 Sales process 5

4.2 Purchase process 6

5 Class Diagram **8**

6 Database Diagram **9**

7 Design Changes from the Proposal **10**

1 Introduction

This document presents the analysis and design of the **Inventory Management System** described in the project proposal. It contains the usage scenarios, the use case diagram, two activity diagrams (sales and purchase), the class diagram and the database diagram.

The system follows a **Client–Server–Database** architecture: a React.js client calls RESTful APIs of a Node.js and Express.js server, which stores data in MongoDB. Access is controlled with JWT authentication and three roles: **Administrator**, **Manager** and **Staff**.

2 Scenarios

A scenario describes one realistic situation, step by step, from the user's point of view. The scenarios below cover the main business flows of the system.

2.1 Scenario 1: Walk-in sale

Actor: Staff. **Precondition:** the staff user is logged in; the product is in stock.

1. A customer without an account wants to buy two Bluetooth speakers and pay in cash.
2. The staff user opens *New sale*, leaves the customer empty (walk-in) and selects *Cash*.
3. The staff user searches the product by name, enters quantity 2 and creates a draft sale.
4. The system takes the price from the product record and saves the sale as `DRAFT`.
5. The staff user presses *Complete*. The system checks the stock, reduces it by 2, writes a stock movement and records the full payment.
6. The staff user prints the invoice and gives it to the customer.

Result: the sale is `COMPLETED` and `PAID`; the stock and its history are updated.

2.2 Scenario 2: Credit sale with partial payment

Actor: Staff. **Precondition:** the customer is registered.

1. A regular customer buys a laptop worth 90,000 but pays only 50,000 now.
2. The staff user creates and completes the sale with the customer selected. The sale becomes `UNPAID` with 90,000 outstanding.
3. The staff user collects a payment of 50,000 by mobile banking with the transaction ID as reference.
4. The system saves the payment and changes the status to `PARTIALLY_PAID` with 40,000 outstanding.
5. Later, the Manager opens *Payments and Dues* and sees the customer's due.

Result: the due is tracked against a named customer; a payment can never exceed the outstanding amount.

2.3 Scenario 3: Purchase from a supplier and supplier payment

Actor: Manager. **Precondition:** the supplier and products exist.

1. The shop orders 10 speakers from a supplier. The Manager creates a draft purchase; the unit price is filled from the product's purchase price and can be changed.
2. When the goods arrive, the Manager checks them and presses *Complete*. The system increases the stock by 10 and writes a `PURCHASE` stock movement.
3. The Manager pays part of the bill by bank transfer using *Pay supplier*.
4. The purchase shows *Partly paid*; *Payments and Dues* shows how much is still owed to the supplier.

Result: stock, purchase total and supplier dues are all correct. A draft can be cancelled without touching stock; a completed purchase cannot be cancelled.

2.4 Scenario 4: Damaged product and low-stock alert

Actor: Manager.

1. During a stock count, one keyboard is found broken.
2. The Manager opens *Inventory*, chooses *Adjust stock*, type *Damaged / lost*, quantity 1 and writes a reason.
3. The system reduces the stock (never below zero) and writes the movement with the reason and user.
4. The stock is now at the minimum level, so the product appears in the low-stock alert on the dashboard.

2.5 Scenario 5: Customer return, and cancelling a wrong return

Actor: Manager.

1. A customer returns one item from an unpaid sale. The Manager records a customer return with a reason. Stock increases and the customer's balance is reduced by the item's value.
2. The Manager notices the return was saved against the wrong sale and cancels it with a reason.
3. In one transaction the system takes the item out of stock again, restores the balance and marks the return `CANCELLED` (the record is kept).

Note: a return that already handed cash back to the customer cannot be cancelled.

2.6 Scenario 6: Administrator adds a staff user

Actor: Administrator.

1. A new salesperson joins. The Administrator creates a user with the `STAFF` role and a temporary password.
2. At the first login the new user must change the password.
3. The staff user can sell and add customers, but cannot see purchases, reports, user management or the audit log.
4. Every important action is written to the audit log, which only the Administrator can view.

3 Use Case Diagram

Figure 1 shows what each role can do. The roles form a hierarchy: a Manager can do everything a Staff user can, and an Administrator can do everything a Manager can. The diagram follows the role checks that the server applies on every API request.

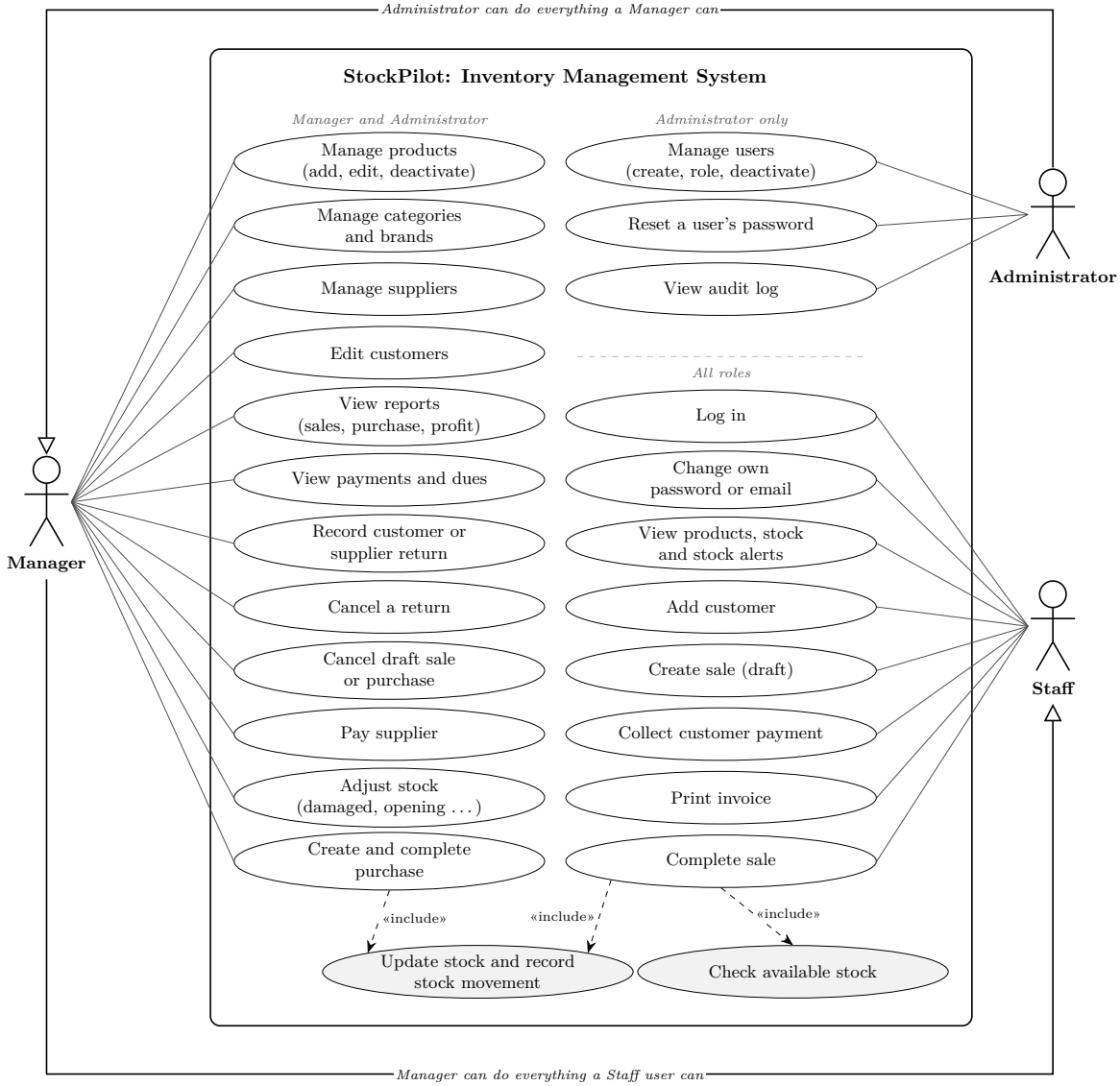


Figure 1: Use case diagram

Table 1: Actors and their responsibilities

Actor	Responsibilities
Staff	Log in, view products and stock, create and complete sales, collect customer payments, print invoices and add customers.
Manager	All Staff tasks, plus products, categories, brands, suppliers, purchases, supplier payments, stock adjustment, returns, cancelling drafts and returns, reports, and payments and dues.
Administrator	All Manager tasks, plus user management (create, role, deactivate, reset password) and the audit log.

4 Activity Diagrams

The activity diagrams show the two most important business processes. The left lane shows what the user does and the right lane shows what the system does. The dashed box marks the steps that run inside one MongoDB transaction: either all of them are saved, or none of them.

4.1 Sales process

A sale is first saved as a draft. When it is completed, the system checks the stock of every item. If any item does not have enough stock, the whole transaction is rolled back. A walk-in customer pays the full amount at completion; a named customer can pay later in parts.

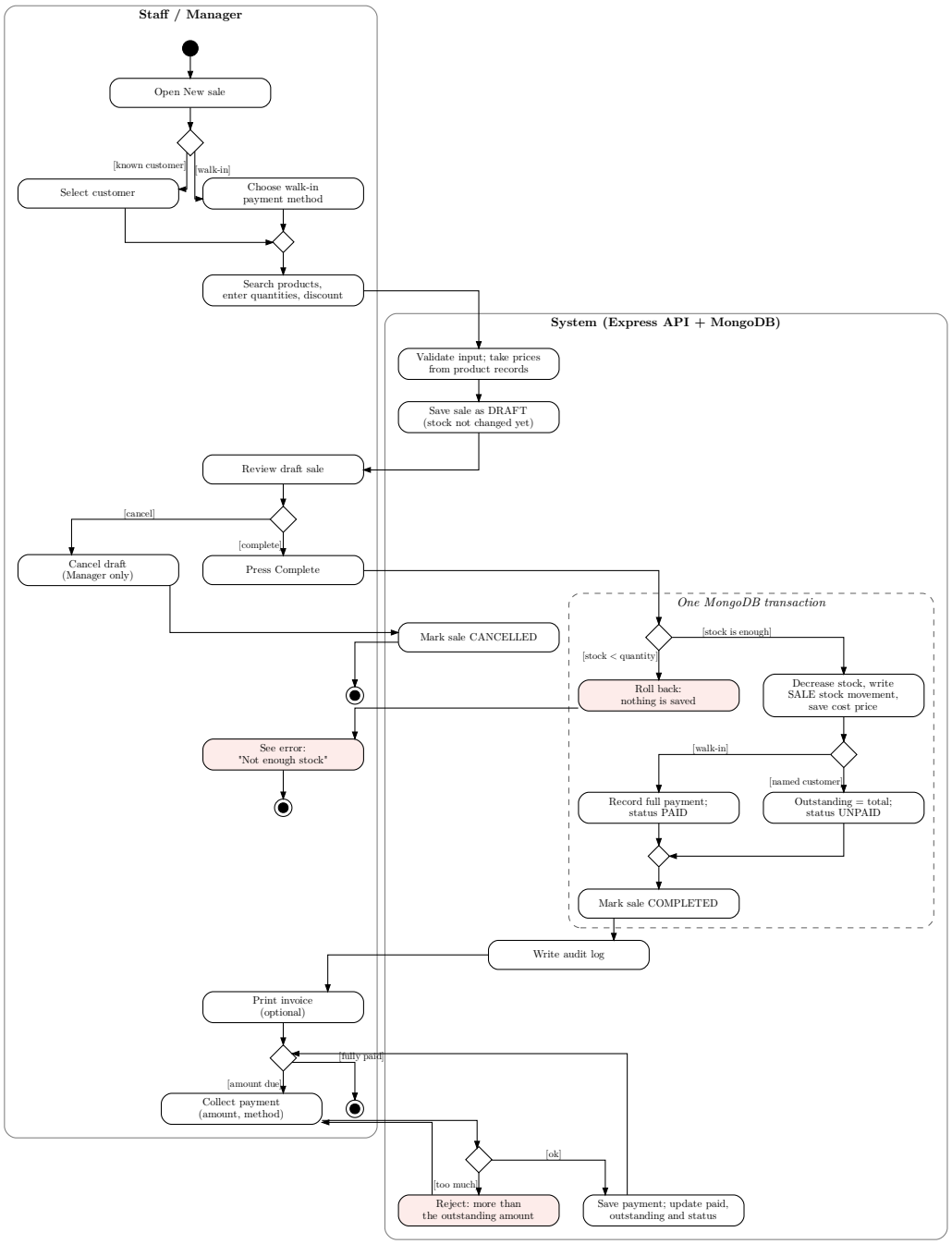


Figure 2: Activity diagram: sales process

4.2 Purchase process

A purchase is also a draft first. Completing it increases the stock and writes the stock history in one transaction. Payments to the supplier can be made at once or later, and can never exceed what is still owed (bill minus returned goods minus payments already made).

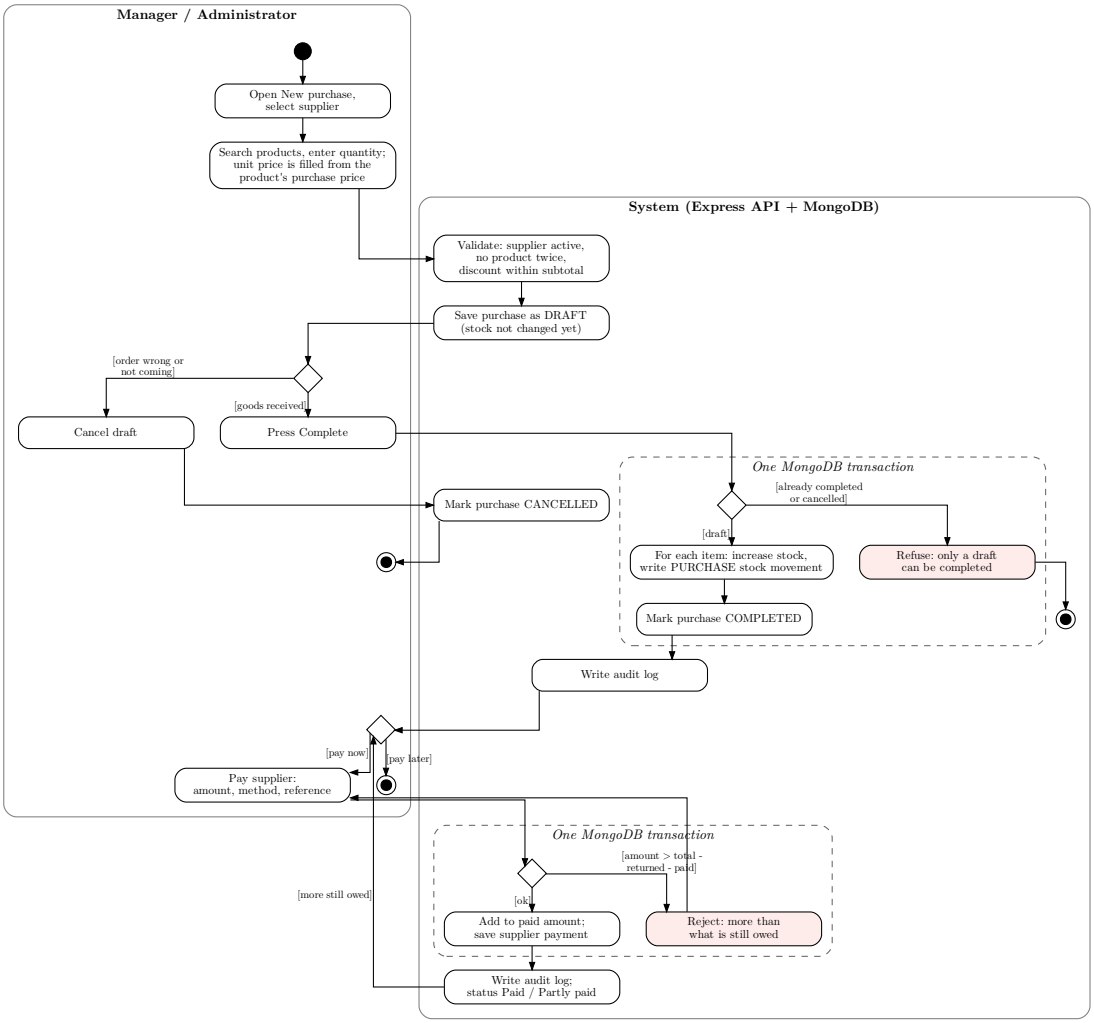


Figure 3: Activity diagram: purchase process

5 Class Diagram

Each class is a Mongoose model in `server/src/models`. Sale, purchase and return lines are *embedded documents* (filled diamond = composition): they are stored inside their parent document and cannot exist alone. Enumerations list the allowed values of status and type fields.

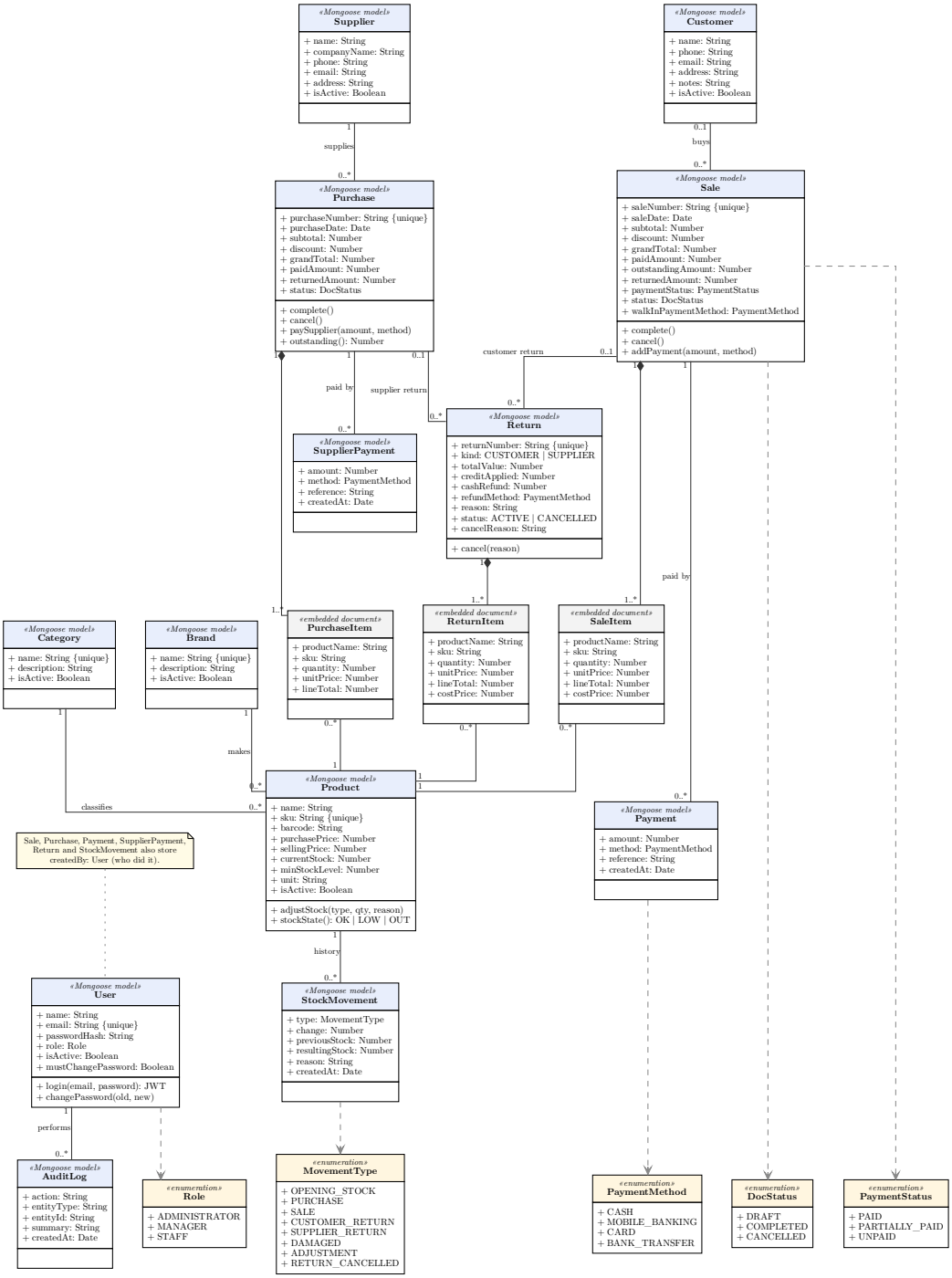


Figure 4: Class diagram

6 Database Diagram

The database is MongoDB with 13 collections. References between collections are stored as `ObjectId` values (FK). Line items are embedded inside sales, purchases and returns, so one document holds a whole bill and can be read in one query.

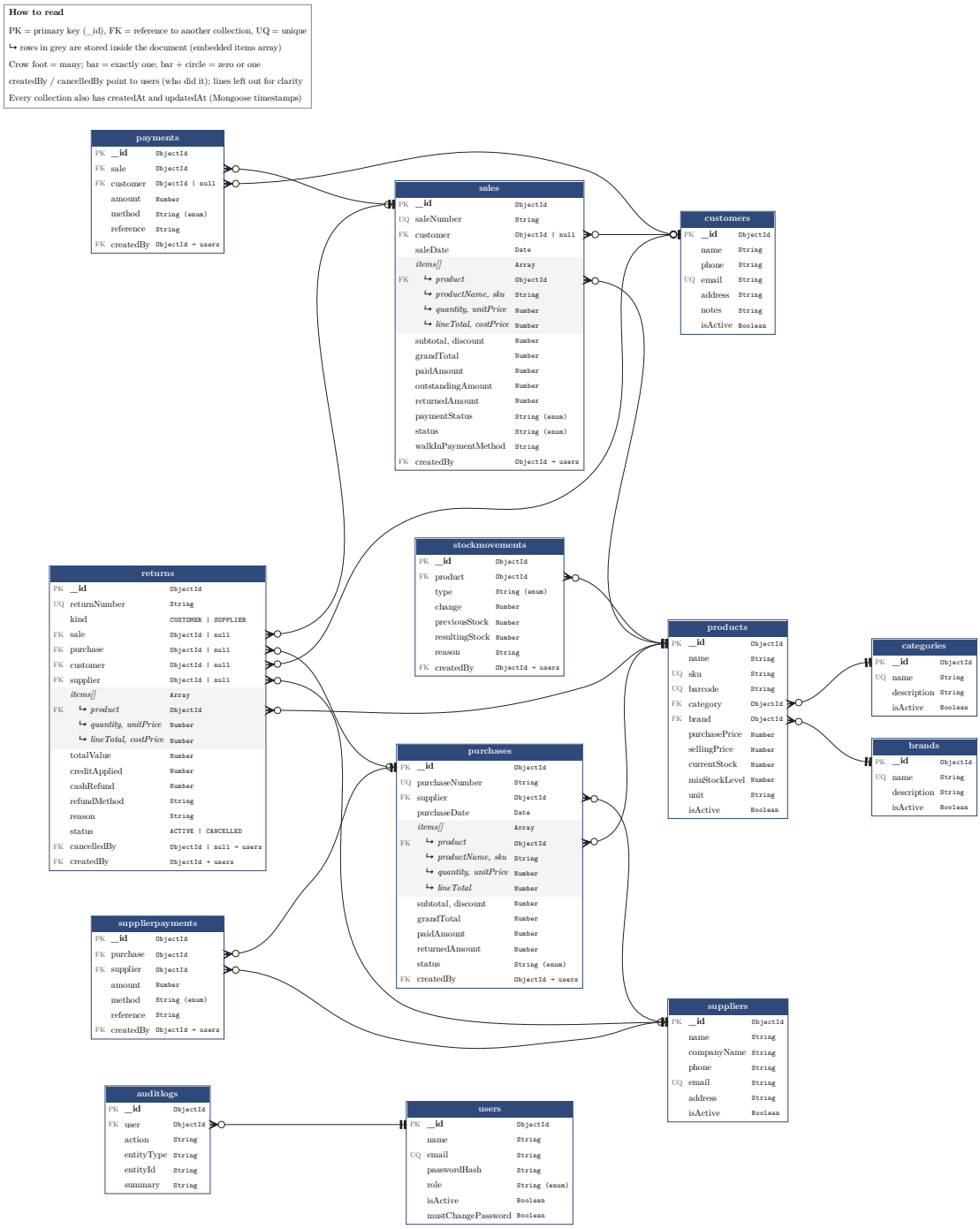


Figure 5: Database (ER) diagram of the MongoDB collections

Table 2: Collections

Collection	Stores
users	System users, hashed passwords and roles
categories, brands	Product classification
products	Products, SKU, barcode, prices, current stock, minimum stock level
customers, suppliers	Business partners
purchases	Purchase bills with embedded items, paid and returned amounts
sales	Sales bills with embedded items, cost price per line, payment status
payments	Money received from customers for sales
supplierpayments	Money paid to suppliers for purchases
returns	Customer and supplier returns with embedded items; can be cancelled
stockmovements	Every stock change with type, before and after quantity, reason and user
auditlogs	Who did what and when

7 Design Changes from the Proposal

The design was refined while building the system, as the proposal allowed (*"The final database schema and relationships will be refined during the system design phase"*).

Table 3: Proposal compared with the final design

Topic	Proposal	Final design and reason
Roles	Separate `roles` collection	A `role` field on `users` with three fixed values. The roles never change at run time, so a collection is not needed.
Line items	`purchaseItems`, `saleItems`, `returnItems` collections	Embedded arrays inside `purchases`, `sales` and `returns`. A bill and its lines are always read and saved together, which MongoDB handles best as one document.
Invoices	`invoices` collection	The printable invoice is generated from the sale or purchase, so no copy of the data is stored.
Payments	One `payments` collection	`payments` (from customers) and `supplierpayments` (to suppliers), because the two have different rules and links.
Batch and expiry	`batches` collection	Not in this version. Planned as future work.
Returns by Staff	Staff can handle returns	Only Manager and Administrator, because a return changes stock and money.